

Your tests pass. That is not the same as your tests working.

TestGuard takes the promises your code makes, breaks each one on purpose, and reports every promise your tests did not notice breaking.

THE PROBLEM

Coverage tells you a line ran. It never tells you anyone checked the result. So a codebase can be fully covered and completely undefended — and nothing in CI will say a word.

```
A REAL TEST, FROM THIS PROJECT'S OWN FIXTURE
expect(store.writeAudit).toHaveBeenCalledWith(
  expect.objectContaining({ action: 'MASK', scope: 'g1', ruleCount: 1 }),
); // `content` is never named — so nothing checks it
```

`objectContaining` ignores keys it does not list. Swap the redacted text for the **raw secret** and this test still passes. Coverage of that line: 100%. The audit log now leaks the very thing it exists to protect.

This shape is disproportionately common in AI-written tests, which are fluent, plausible, and pin down far less than they appear to.

THE METHOD

Break the code deliberately. Watch what the tests do.

killed

You broke it — a test failed. **Good.** That behaviour is genuinely defended.

SURVIVED

You broke it — everything stayed green. **A blind spot.**

The word trips everyone once: it is the *fault* that survived, not the test. SURVIVED is the bad news. A healthy report is full of *killed*.

A tool that reports everything as caught is worse than no tool at all — because nobody questions good news.

THE DESIGN CONSTRAINT EVERYTHING ELSE FOLLOWS FROM

HOW A RUN WORKS

- 1 **You state a claim.** Plain language, in a file: “*A missing scope fails closed.*” “*The audit row never contains the original input.*”
- 2 **Each claim carries faults** — concrete edits that would make that promise false.
- 3 **Probe isolates.** Your repository is copied to a scratch worktree. Your working tree is never touched.
- 4 **Baseline, three times.** The claim's tests run on *unmodified* code and must pass all three. If they cannot, nothing about that claim can be trusted, and the run says so instead of guessing.
- 5 **Inject, run three times, restore.** The fault goes in, the same tests run again, the file is put back.
- 6 **Read the structured report — never the exit code.** An exit code cannot tell “a test failed” apart from “nothing loaded.”
- 7 **Write evidence.** One verdict per fault, with the runs that produced it.

SEVEN VERDICTS, BECAUSE “DID THE TESTS FAIL?” IS A SLOPPY QUESTION

A red suite is not proof of detection. Most of the design is refusing to conflate these:

VERDICT	WHAT IT MEANS
killed	A test body ran and rejected the behaviour. The only good outcome.
SURVIVED	Everything passed. A real blind spot in the tests.
NOCOVER	No test even looks at this code. Not “weak tests” — <i>no</i> tests.
UNVERIFIABLE	The fault could not be applied: its anchor moved, or matches twice. Nothing was learned.
FAULT-INVALID	The break itself was broken — it did not compile. Our fault, not yours.
TIMEOUT	The suite hung. A hang is not a detection.
FLAKY-DEFENDER	The tests are not reliable enough on untouched code to be asked the question.

EVERY AMBIGUITY ROUNDS TOWARD “UNPROVEN”

- Three runs, never one.
- A timeout is never a kill.
- Mixed results across runs are not a kill.
- A green baseline is required before any fault is injected.
- A suite that fails to load is never a kill.
- A test that mocks what it defends is not a defender.

WHAT IS NOT NEW

Breaking code to test your tests is **mutation testing**, and it dates to the 1970s. Stryker, PIT, mutmut and Cosmic Ray all do it. Anyone claiming that part is novel is selling something.

WHAT IS DIFFERENT: THE QUESTION BEING ASKED

CLASSIC MUTATION TESTING

Mutates everything, mechanically

“How good are my tests?”

→ **Mutation score: 73%**

A number. Not actionable, not auditable, and it cannot tell you which promise is at risk.

TESTGUARD

Binds every fault to a stated claim

“Is *this specific promise* defended?”

→ **“Your project says a missing scope fails closed. Nothing checks that.”**

A finding. An assessor reads your claims — reviewable like code — not a score.

That is the difference between a **metric** and an **audit**, and it is the whole design.

THREE MORE THINGS THAT FOLLOW FROM IT

Paranoid about itself

Most tools mark a mutant killed when the test command exits non-zero — conflating “a test caught it” with “everything exploded.” Here, the verdict set is closed and each member means something different.

Evidence, not output

Schema-validated JSON with stable fingerprints. Freeze today's blind spots as a baseline, gate CI on *new* ones — so a legacy codebase can adopt it on Monday without four hundred failures.

Built for AI-written code

When an agent writes the tests, the failure mode is not “no tests.” It is confident, plausible tests that assert nothing. That is precisely what this detects.

Coverage tells you a line ran. TestGuard tells you which of your promises nobody is actually defending.

[GITHUB.COM/RACCIOLY/TESTGUARD](https://github.com/raccioly/testguard) · [NPM TESTGUARD-CLI](https://www.npmjs.com/package/testguard-cli) · [MIT](https://mit.edu)